

Week 2:

Optimistic developer: "This is a common sense feature that doesn't need documentation"

End user:



One Page Web-Apps and Bootstrap

Chapters 2, 3

INFT3050 – Web Programming

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Newcastle in accordance with section 113P of the *Copyright Act 1968* (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

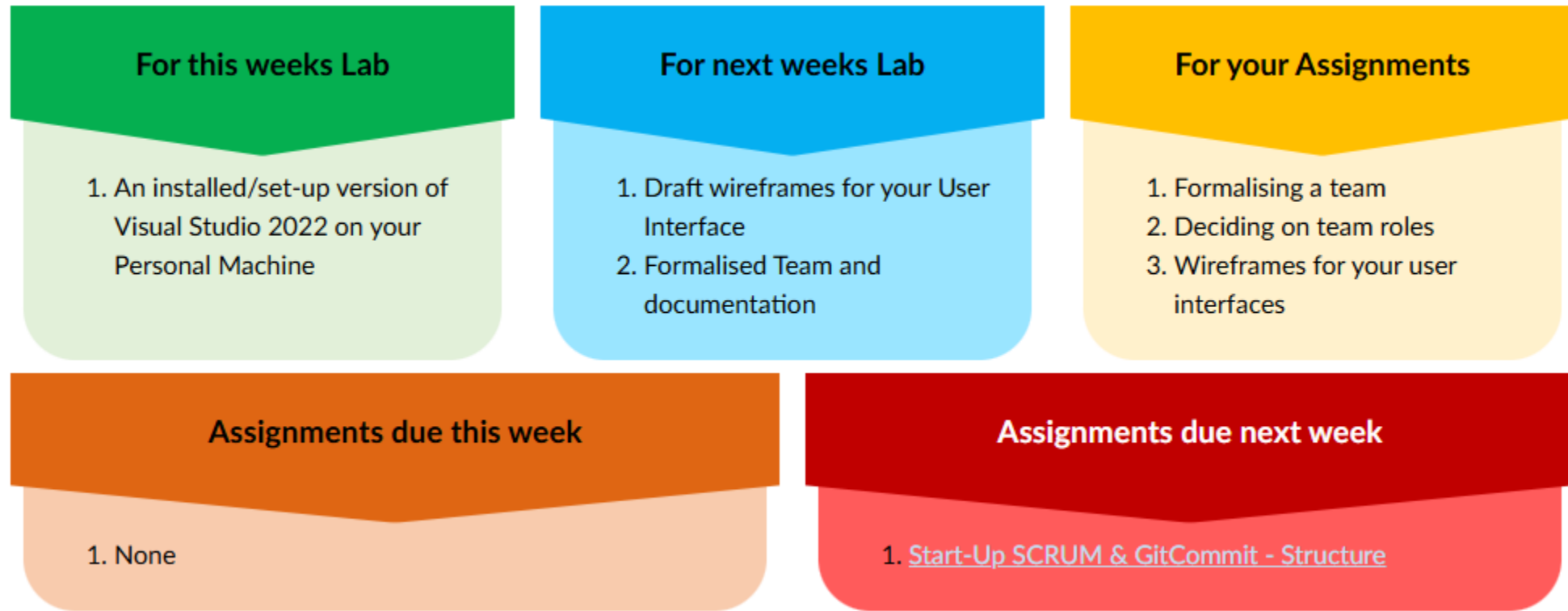
Do not remove this notice



Contents

- Housekeeping
- One-Page Web Apps
- Bootstrap for your Web App
- Demos

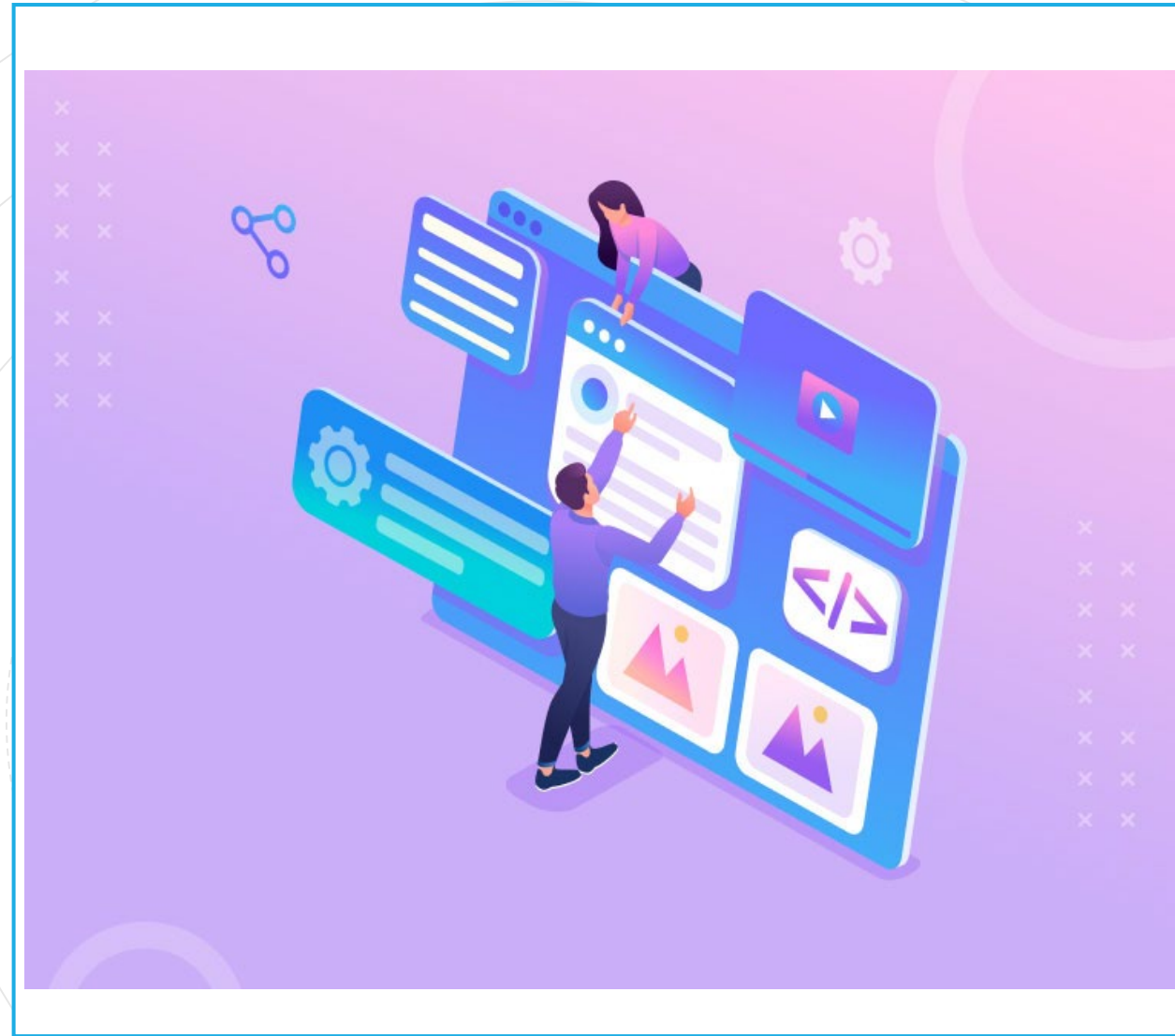
Housekeeping



Project Update

1. All documents live in Canvas, please read these
2. Forming Groups (if you are working in a group)
3. If you choose to work in a group – please complete the Group Code of Conduct
4. Start wireframing your web-apps Views (user interface)
5. Please ask questions to me or Connor to clarify anything about this project

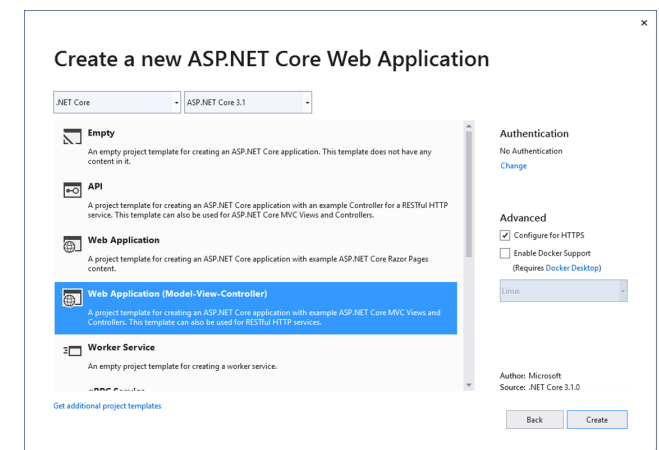
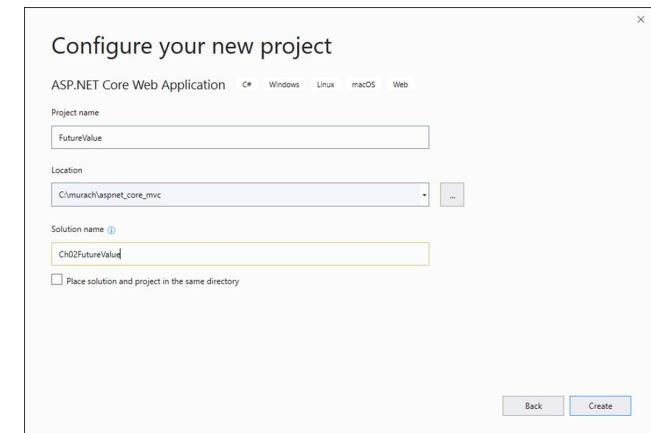




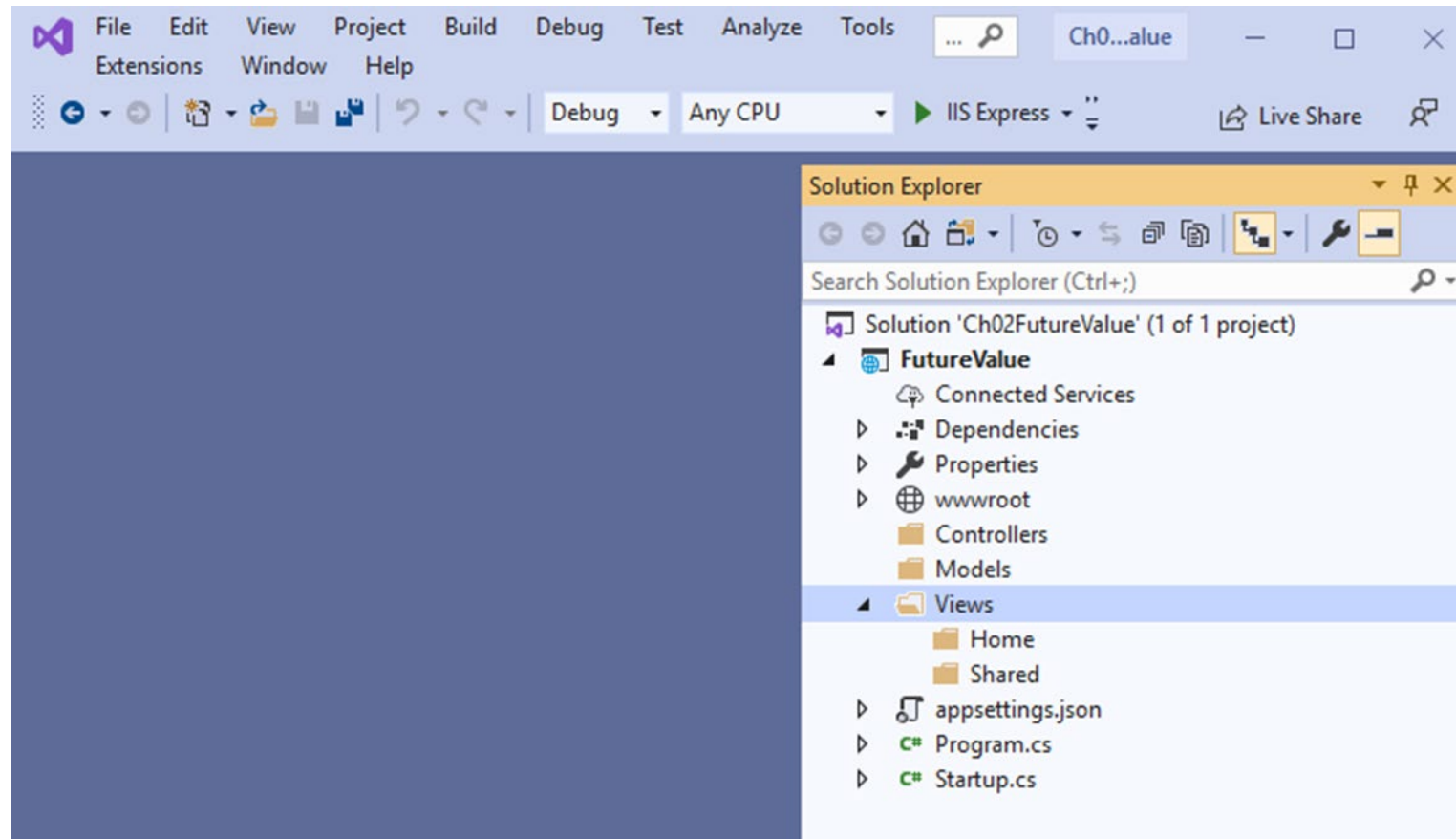
One-Page Web Apps

How to create a new Web-App

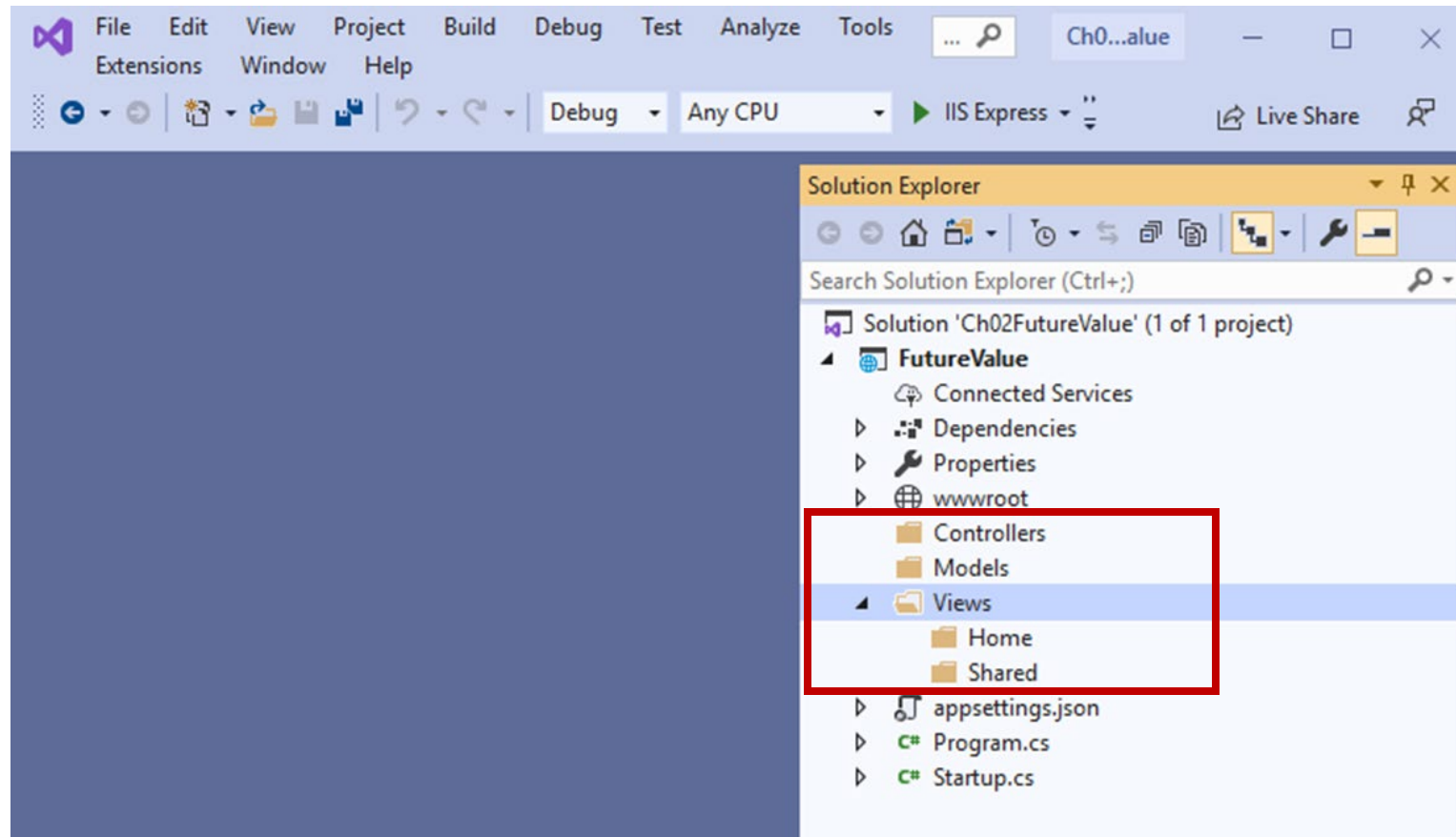
- Start Visual Studio.
- From the menu system, select File→New→Project.
- Select the ASP.NET Core Web Application item and click the Next button.
- Enter a project name.
- Specify the location (folder). To do that, you can click the Browse button.
- If necessary, edit the solution name and click the Create button.
- Use the resulting dialog to select the Web Application (Model-View-Controller) template or the Empty template.



Visual Studio with the folders for an MVC Web-App



Visual Studio with the folders for an MVC Web-App



Add and Remove components in the solution

Add

- Select the folder (for example Views)
- Right click → Add → New Folder/Controller/Other item
- NOTE: it is good practice to organise the newly added items e.g., Controllers go in the Controllers Folder

Delete

- Expand the folder
- Either right click and select delete OR press delete on your keyboard
- DO NOT delete the folder

What does a Controller do?

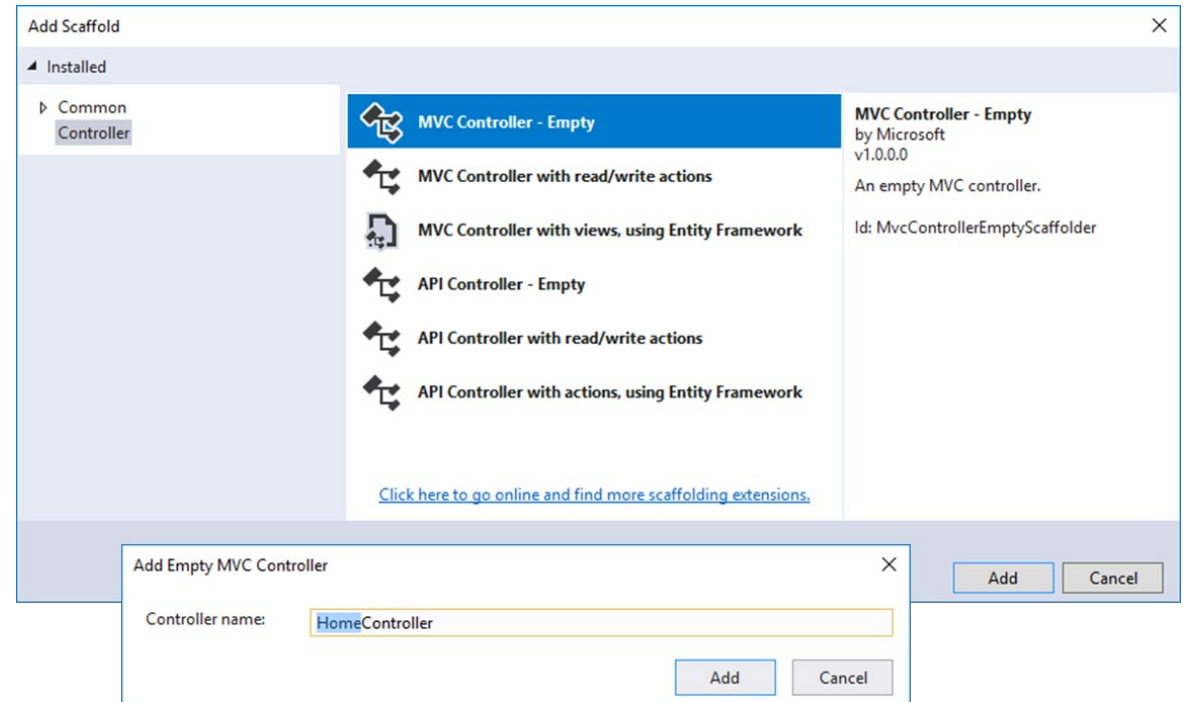
- A **controller** is a C# class that inherits from the Controller class that's available from the `Microsoft.AspNetCore.Mvc` namespace.
- When a controller runs in response to HTTP action verbs like GET or POST this is called an **action method** or **action**
- When the `Index()` action has some stored some data in a **ViewBag** it uses the `View()` method to return the `ViewResult` object for that action

```
using Microsoft.AspNetCore.Mvc;

namespace FutureValue.Controllers
{
    public class HomeController :
Controller
    {
        public IActionResult Index()
        {
            ViewBag.Name = "Mary";
            ViewBag.FV = 99999.99;
            return View();
        }
    }
}
```

How to add a Controller

- Right-click the Controllers folder and select Add→Controller.
- In the Add Scaffold dialog, select “MVC Controller – Empty” and click Add.
- In the Add Empty MVC Controller dialog, name the controller and click Add.



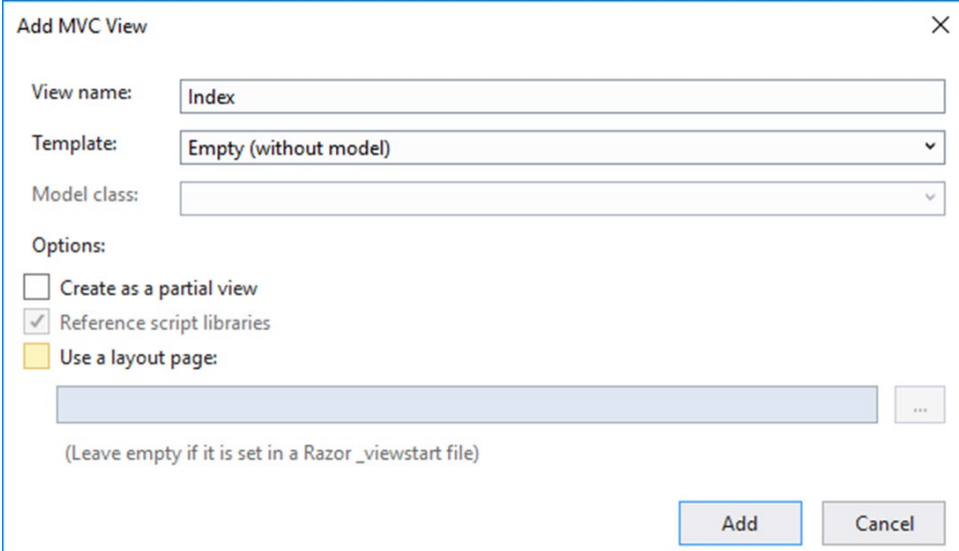
What is a Razor view?

- A **Razor view** contains both C# and HTML code. That's why its file extension is `.cshtml`
- To execute one or more C# statements, you can declare a Razor code block by coding the `@` sign followed by a pair of braces `{ }`

```
@{  
    Layout = null;  
}  
<!DOCTYPE html>  
<html>  
  <head>  
    <meta name="viewport"  
    content="width=device-width" />  
    <title>Home Page</title>  
  </head>  
  <body>  
    <h1>Future Value Calculator</h1>  
    <p>Customer Name: @ViewBag.Name</p>  
    <p>Future Value:  
    @ViewBag.FV.ToString("C2")</p>  
  </body>  
</html>
```

How to add a view to the Views/Home folder

- In the Solution Explorer, right-click the Views/Home folder and select Add→View.
- In the resulting dialog, enter Index as the name of the view.
- If necessary, select the “Empty (without model)” template.
- Deselect the “Use a layout page” checkbox.
- Click the Add button.



The screenshot shows the 'Add MVC View' dialog box. It has a title bar with a close button (X). The dialog contains the following fields and options:

- View name:** A text box containing 'Index'.
- Template:** A dropdown menu showing 'Empty (without model)'.
- Model class:** A dropdown menu.
- Options:**
 - ☐ Create as a partial view
 - ☒ Reference script libraries
 - ☐ Use a layout page: A text box with a blue background and an ellipsis button (...).

Below the 'Use a layout page' option, there is a note: '(Leave empty if it is set in a Razor _viewstart file)'. At the bottom right, there are two buttons: 'Add' and 'Cancel'.

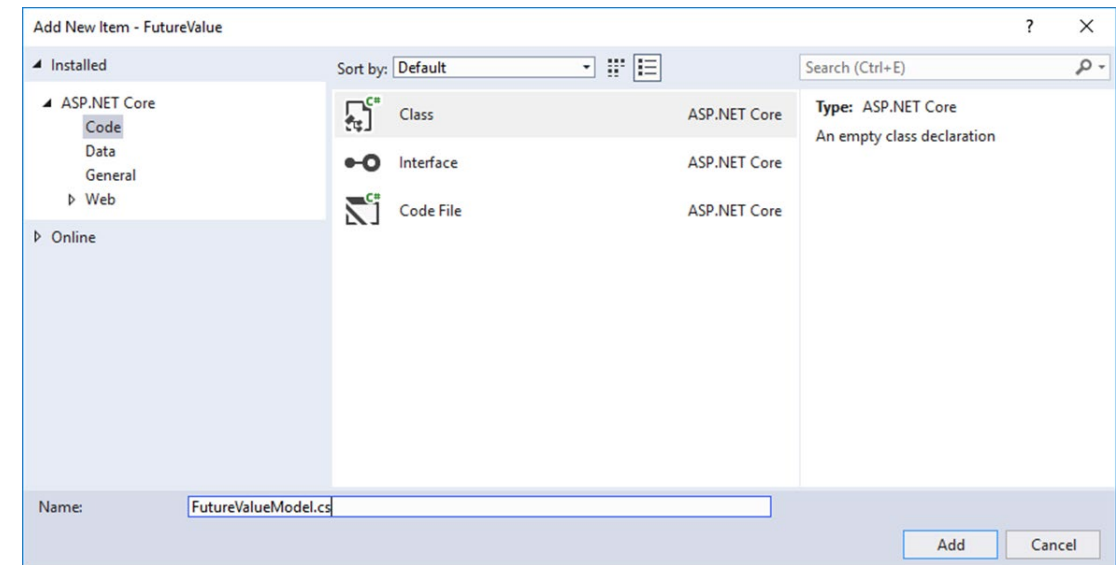
What is a model?

- A **model** is a regular C# class that defines the data objects for a page and is typically stored in the Models folder.
- This sample code has 3 properties that are used to get-and-set the monthly investment, yearly interest rate and number of years
- The sample code also has a basic loop to calculate future value

```
namespace FutureValue.Models
{
    public class FutureValueModel
    {
        public decimal MonthlyInvestment { get; set; }
        public decimal YearlyInterestRate { get; set; }
        public int Years { get; set; }
        public decimal CalculateFutureValue()
        {
            int months = Years * 12;
            decimal monthlyInterestRate =
                YearlyInterestRate / 12 / 100;
            decimal futureValue = 0;
            for (int i = 0; i < months; i++)
            {
                futureValue = (futureValue + MonthlyInvestment) *
                    (1 + monthlyInterestRate);
            }
            return futureValue;
        }
    }
}
```

How to add a file for a model class

- In the Solution Explorer, right-click the Models folder and select Add→Class.
- In the resulting dialog, enter the name of the class, and click the Add button.

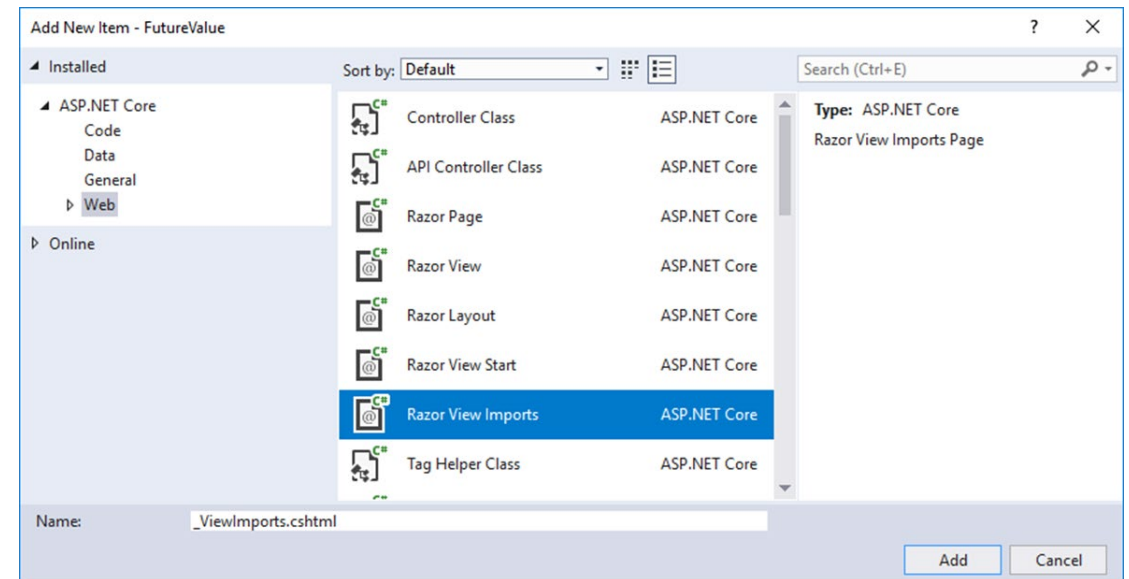


What's a Razor view import page?

- A Razor view imports page makes it easier to work with models and tag helpers. As a result, most web apps include this page

How to add a Razor view imports page

- In the Solution Explorer, right-click the Views folder and select Add→New Item.
- In the resulting dialog, select the Installed→ASP.NET Core→Web category, select the Razor View Imports item, and click the Add button.



How to code a strongly-typed view

- You use the `@model` directive to **bind** the model to the view. This kind of view is called a **strongly-typed view**.
- A razor view import page makes it easier to work both **model classes** and **tag helpers**

Common Tag helper	HTML tags
<code>asp-for</code>	<code><label></code> <code><input></code>
<code>asp-action</code>	<code><form></code> <code><a></code>
<code>asp-controller</code>	<code><form></code> <code><a></code>

How to handle GET and POST requests

- When an action method accepts a model object as an argument, MVC uses the data stored in the GET or POST request to set the properties of the model object.
- As a result, the action method can use the model object to work with the posted data.
- In addition, the POST version of the Index() method can use the View() method to pass the model on to the view.

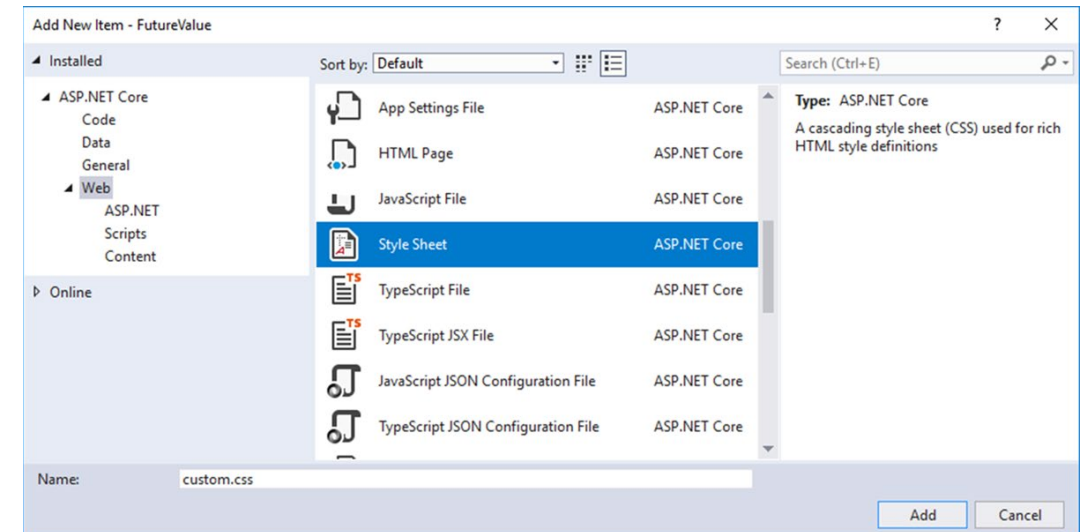
```
using Microsoft.AspNetCore.Mvc;
using FutureValue.Models;

public class HomeController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        ViewBag.FV = 0;
        return View();
    }

    [HttpPost]
    public IActionResult Index(FutureValueModel model)
    {
        ViewBag.FV = model.CalculateFutureValue();
        return View(model);
    }
}
```

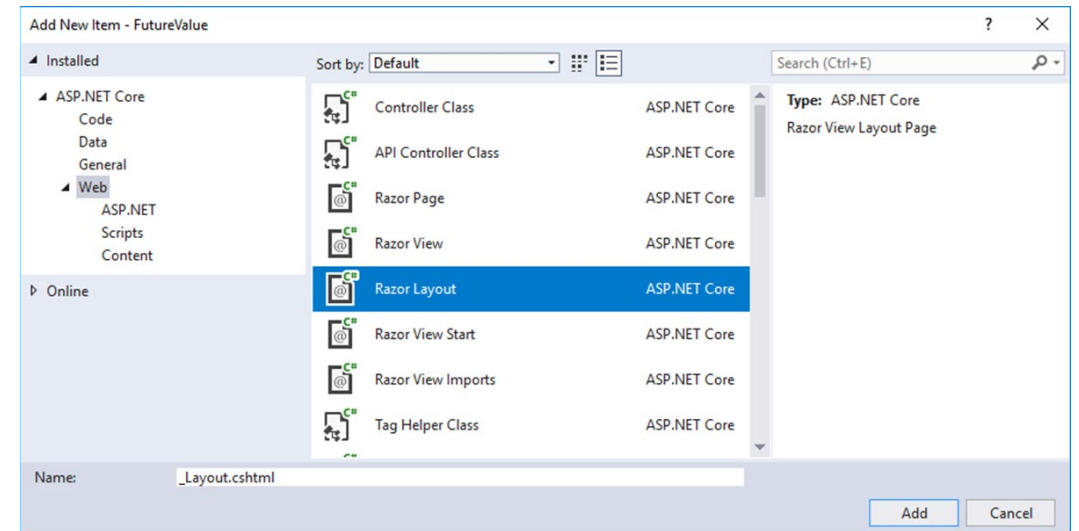
How to add a CSS style sheet

- If the wwwroot/css folder doesn't exist, create it.
- Right-click the wwwroot/css folder and select Add→New Item.
- Select the ASP.NET Core→Web category, select the Style Sheet item, enter a name for the CSS file, and click the Add button.



How to add a Razor layout

- Right-click the Views/Shared folder and select Add→New Item.
- Select the ASP.NET Core→Web category, select the Razor Layout item, and click the Add button.



How to Razor components

- A **Razor layout** provides a way to store elements that are common to multiple web pages in a single file.
- A **Razor view start** lets you specify the default Razor layout for the Razor views of a web app.

How to add a Razor...

Razor View Start

- Right-click the Views folder (not the Views/Shared folder) and select Add→New Item.
- Select the ASP.NET Core→Web category, select the Razor View Start item, and click the Add button.

Razor View

- Right-click the folder for the view (Views/Home, for example) and select Add→View.
- If you're using a layout that has a view start, select the "Use a layout page" item but don't specify a name for the layout page.

How to add a Razor...

Razor View Start

- Right-click the Views folder (not the Views/Shared folder) and select Add→New Item.
- Select the ASP.NET Core→Web category, select the Razor View Start item, and click the Add button.

Razor View

- Right-click the folder for the view (Views/Home, for example) and select Add→View.
- If you're using a layout that has a view start, select the "Use a layout page" item but don't specify a name for the layout page.

How to add a Razor view when using a layout with a view start

- You can add a Razor view that works with the Razor layout and view start
- To do that, you select the Razor View item from the Add New Scaffolded Item dialog instead of the Razor View - Empty item as shown earlier.
- Then, the dialog in this figure is displayed, which lets you name the view and indicate that you want to use a layout page with it.

Validation – Protect Users from themselves and protect your organisations data

- Validation protects:
 - Users from entering in correct data
 - Your web-app from crashing
 - Your organisations' data storage systems from having unusable data (data that can later be used for analysis)

Future Value Calculator

https://localhost:5001

Future Value Calculator

- Please enter a monthly investment.
- Yearly interest rate must be between 0.1 and 10.0.
- Number of years must be between 1 and 50.

Monthly Investment:

Yearly Interest Rate:

Number of Years:

Future Value:

[Clear](#)

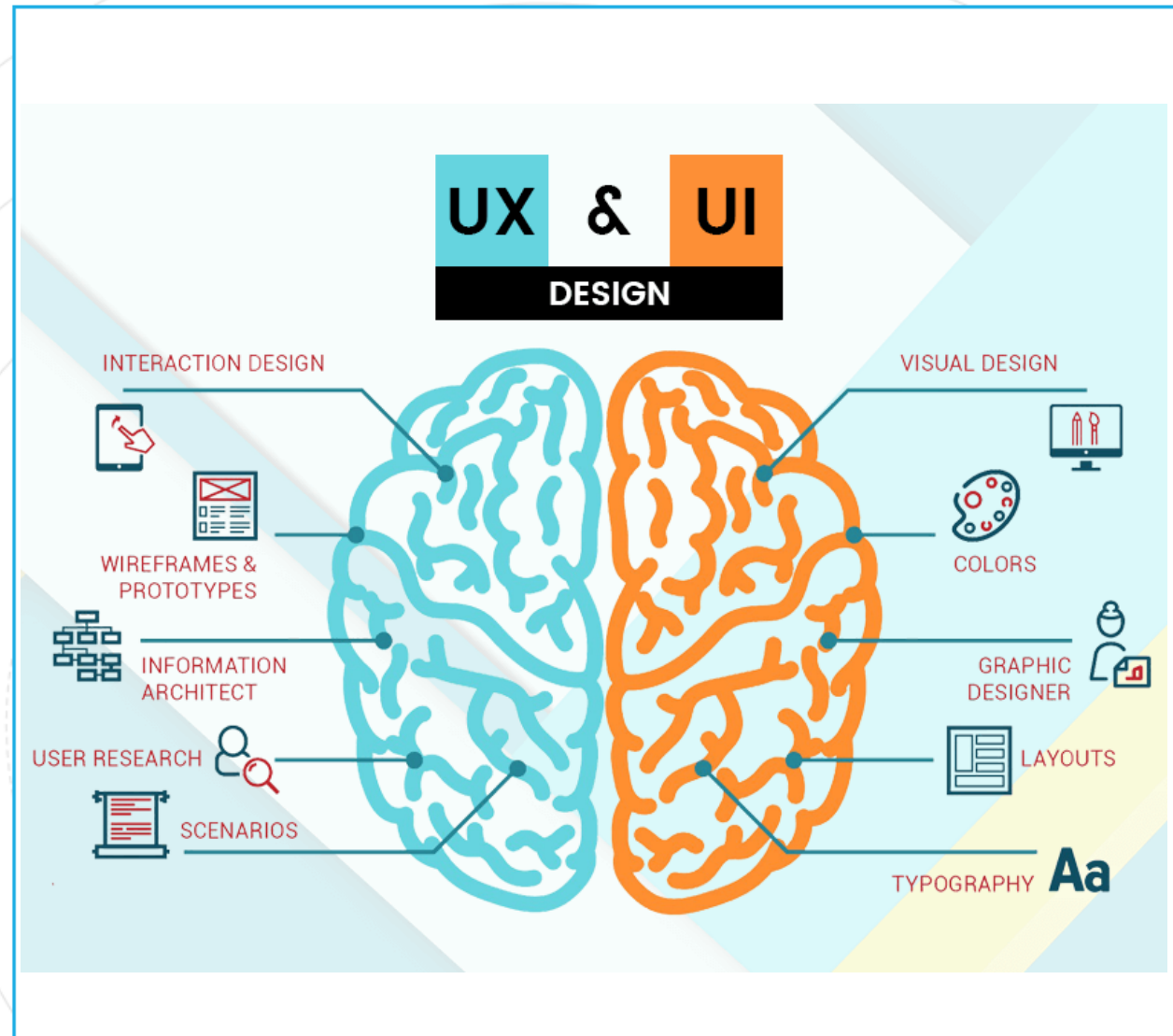
Looking for this code?

- The full working code is available on Canvas with some handy comments
- We will run through this development in our Labs this week
- I've also made a handy cheat sheet of the terms in the first half of the lecture



MVC Cheat Sheet

Folder	Name	Used For
Controller	Controller	A C# class that inherits from the Controller class that's available from the Microsoft.AspNetCore.Mvc namespace. When you develop an MVC app, it's common to place controller classes in a namespace that consists of the project name, a dot, and the name of the folder that stores the controllers.
Model	Model	A regular C# class that defines the data objects for a page and is typically stored in the Models folder.
View	Razor Layout	Store elements that are common to multiple web pages in a single file
View	Razor Start View	Lets you specify the default Razor layout for the Razor views of a web app
View	Razor View	Provides a way to store elements that are unique to a web page both C# and HTML code
View	Razor View Imports Page	Makes it easier to work with your model classes and the tag helpers that are available from ASP.NET Core
wwwroot/css	CSS Style Sheet	A way to store the formatting for multiple web pages in a single external file.



User Interface Design Guidelines

User interface guidelines: 10 essential rules to follow

1. Visibility of system status
2. Match between the system with the real world
3. Consistency and standards
4. User control and freedom
5. Recognition rather than recall
6. Error prevention
7. Consistency and standards
8. Flexibility and efficiency of use
9. Help users recognise, diagnose and recover from errors
10. Help and documentation

Source: <https://www.uxdesigninstitute.com/blog/10-user-interface-guidelines/>

Designing for Web Accessibility

- Provide sufficient contrast between foreground and background
- Don't use colour alone to convey information
- Ensure that interactive elements are easy to identify
- Provide clear and consistent navigation options
- Ensure that form elements include clearly associated labels
- Provide easily identifiable feedback
- Use headings and spacing to group related content
- Create designs for different viewport sizes
- Include image and media alternatives in your design
- Provide controls for content that starts automatically

Source: <https://www.w3.org/WAI/tips/designing/>

Bootstrap

Primary Secondary Success Info Warning Danger

Home Cart

Email:

Password:

Department	Phone Number	Extension
General	555-555-5555	1
Customer Service	555-555-5556	2
Billing and Accounts	555-555-5557	3

Column 1
Column 2
Column 1
Column 2
Column 1
Column 2

Bootstrap Elements Covered

- Buttons and Button Groups
- Buttons Coloured
- Dropdown boxes and Lists
- Log In samples
- Navigation samples
- Tables
- Text

How to add the Bootstrap and jQuery libraries

1. Start Visual Studio and open a project. In the Solution Explorer, expand the wwwroot/lib folder and delete any old Bootstrap or jQuery libraries.
2. In the Solution Explorer, right-click on the project name and select the Add→Client-Side Library item.
3. In the dialog box that appears, type “jquery@”, select “3.3.1” from the list that appears, and click the Install button.
4. Repeat steps 2 and 3 for the library named “twitter-bootstrap@4.3.1”, but change the target location to “www/lib/bootstrap”.
5. Repeat steps 2 and 3 for the library named “popper.js@1.14.7”.
6. Repeat steps 2 and 3 for the library named “jquery-validate@1.19.0”.
7. Repeat steps 2 and 3 for the library named “jquery-validation-unobtrusive@3.2.11”.

A Razor layout that enables client-side libraries

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title>@ViewData["Title"]</title>
  <link rel="stylesheet"
        href="~/lib/bootstrap/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/custom.css" />

  <script src="~/lib/jquery/jquery.min.js"></script>
  <script src="~/lib/popper.js/popper.min.js"></script>
  <script src="~/lib/bootstrap/js/bootstrap.min.js"></script>

  <script src="~/lib/jquery-validate/
              jquery.validate.min.js"></script>
  <script src="~/lib/jquery-validation-unobtrusive/
              jquery.validate.unobtrusive.min.js"></script>
</head>
<body>
  @RenderBody()
</body>
</html>
```

Bootstrap Documentation

<https://getbootstrap.com/docs/>

Slide vs. IDE

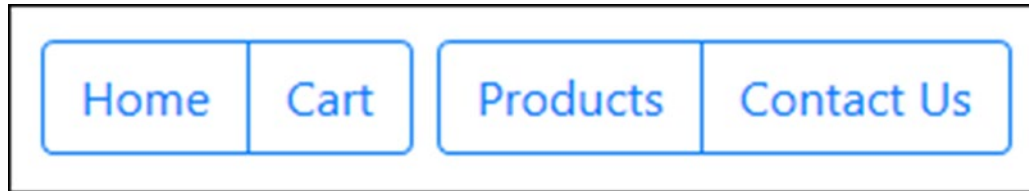
- All code presented in the coming slides is also available in a Visual Studio Solution
- **See** `BootstrapSampleCode` on Canvas

A Basic Button Group



```
<div class="btn-group" role="group"
    aria-label="Button group">
  <a href="/" class="btn btn-outline-primary">Home</a>
  <a href="/cart" class="btn btn-outline-primary">
    Cart</a>
  <a href="/products" class="btn btn-outline-primary">
    Products</a>
  <a href="/contact-us" class="btn btn-outline-primary">
    Contact Us</a>
</div>
```

A toolbar with two button groups



```
<div class="btn-toolbar" role="toolbar"
    aria-label="Toolbar with groups">
  <div class="btn-group mr-2" role="group"
    aria-label="First group">
    <a href="/" class="btn btn-outline-primary">Home</a>
    <a href="/Cart" class="btn btn-outline-primary">Cart</a>
  </div>
  <div class="btn-group" role="group" aria-label="Second group">
    <a href="/products" class="btn btn-outline-primary">
      Products</a>
    <a href="/contact-us" class="btn btn-outline-primary">
      Contact Us</a>
  </div>
</div>
```


Buttons – with Icons

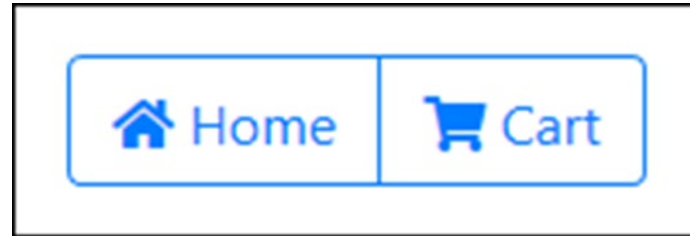
- Font Awesome is our main resource here

<https://fontawesome.com/>

- You will need to install the stylesheet

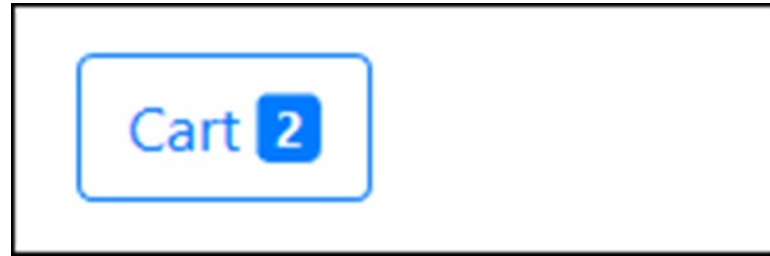
```
<link rel="stylesheet"
      href="https://use.fontawesome.com/releases/v5.8.1/css/all.css"
      integrity="sha-long-hash_code" crossorigin="anonymous">
```

Button Groups – with Icons



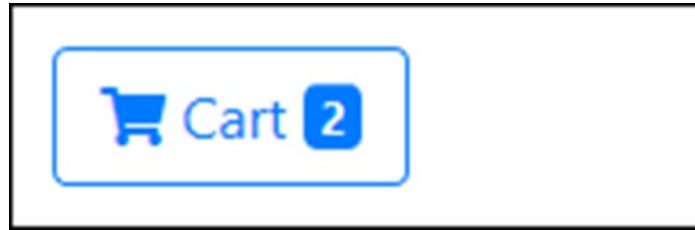
```
<div class="btn-group" role="group"
  aria-label="Button group">
  <a href="/" class="btn btn-outline-primary">
    <span class="fas fa-home"></span>&nbsp;Home&nbsp;
  </a>
  <a href="/cart" class="btn btn-outline-primary">
    <span class="fas fa-shopping-cart"></span>
    &nbsp;Cart&nbsp;
  </a>
</div>
```

Button – with a Badge



```
<a href="/cart" class="btn btn-outline-primary">  
  &nbsp;&nbsp;&nbsp;Cart&nbsp;&nbsp;&nbsp;  
  <span class="badge badge-primary">2</span>  
</a>
```

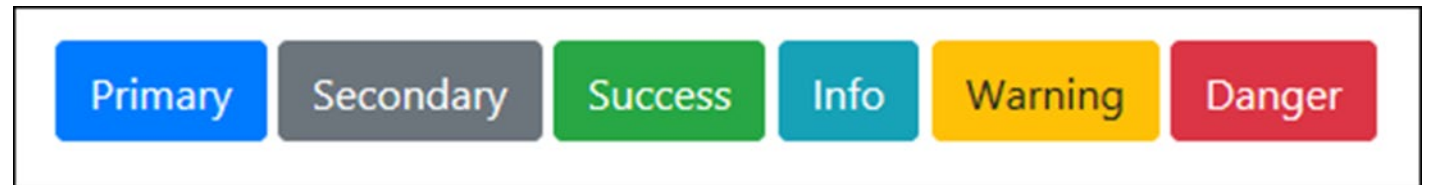
Button – with a Badge and an Icon



```
<a href="/cart" class="btn btn-outline-primary">  
  <span class="fas fa-shopping-cart">  
  </span>&nbsp;Cart&nbsp;  
  <span class="badge badge-primary">2</span>  
</a>
```

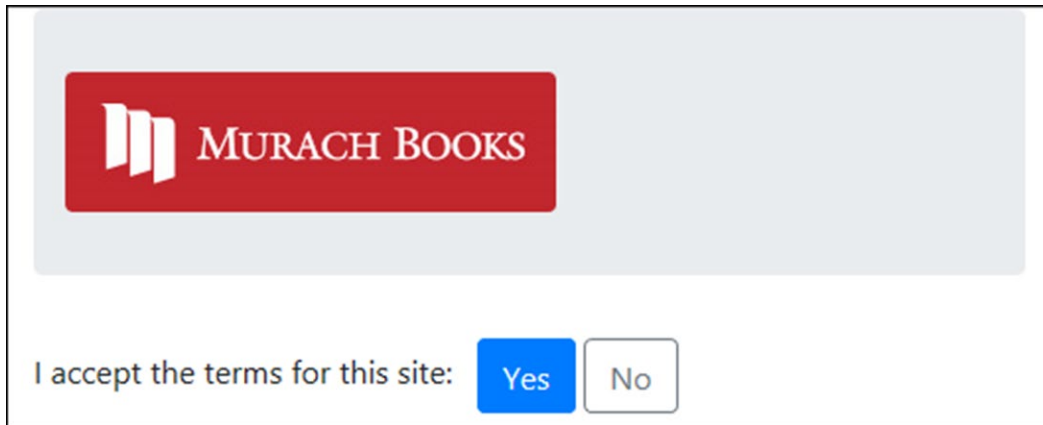
Buttons Coloured Samples

Class	Default color
primary	Dark blue
secondary	Gray
success	Green
info	Light blue
warning	Orange
danger	Red
light	White
dark	Gray



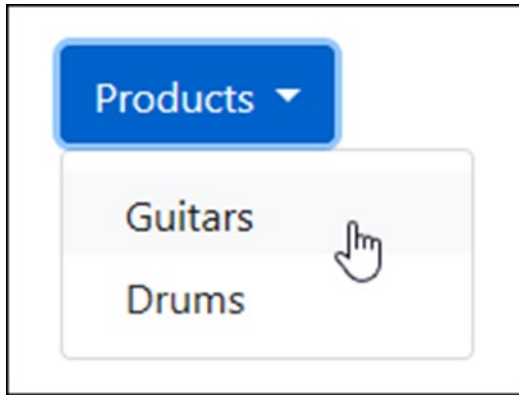
```
<button class="btn btn-primary">Primary</button>  
<button class="btn btn-secondary">Secondary</button>  
<button class="btn btn-success">Success</button>  
<button class="btn btn-info">Info</button>  
<button class="btn btn-warning">Warning</button>  
<button class="btn btn-danger">Danger</button>
```

Buttons – with a jumbotron, an image and two buttons



```
<div class="container">
  <header class="jumbotron">
    
  </header>
  <main>
    <form asp-action="Index" method="post">
      <span class="mr-2">I accept the terms for this site:
    </span>
    <button type="submit" class="btn btn-primary">
      Yes
    </button>
    <button id="btnNo" class="btn btn-outline-secondary">
      No
    </button>
  </form>
</main>
</div>
```

Dropdown Boxes



```
<div class="dropdown">
  <button type="button" class="btn btn-primary dropdown-toggle"
    id="productsDropdown" data-toggle="dropdown"
    aria-haspopup="true" aria-expanded="false">Products
</button>
  <div class="dropdown-menu" aria-labelledby="productsDropdown">
    <a class="dropdown-item"
      href="/product/list/guitars">Guitars</a>
    <a class="dropdown-item"
      href="/product/list/drums">Drums</a>
  </div>
</div>
```

A Basic List



```
<ul class="list-group">  
  <li class="list-group-item">Guitars</li>  
  <li class="list-group-item">Basses</li>  
  <li class="list-group-item">Drums</li>  
</ul>
```

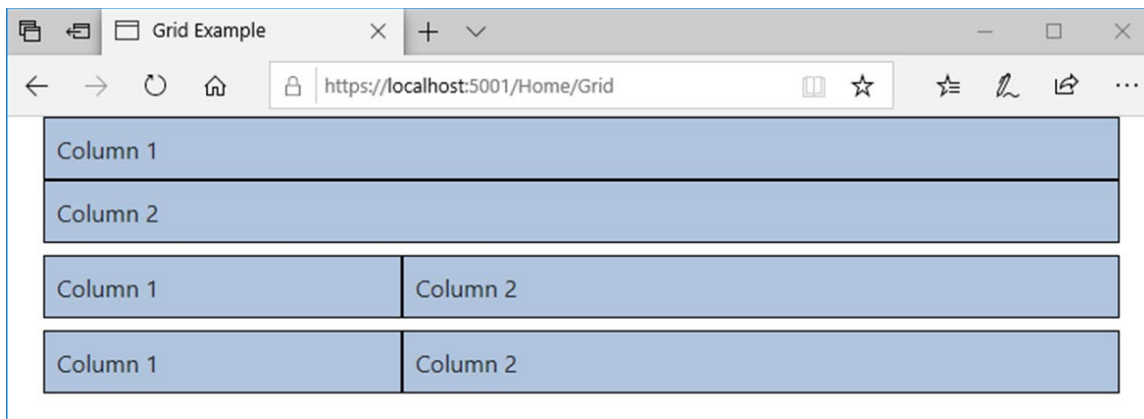

Another basic list group with an active item



```
<div class="list-group">
  <a href="/guitars" class="list-group-item active">
    Guitars</a>
  <a href="/basses" class="list-group-item">Basses</a>
  <a href="/drums" class="list-group-item">Drums</a>
</div>
```

Grid

- Helpful hint: You can change the `class="col-[Number]"` to just `class="col"` for automatic sizing
- You also need to edit the CSS (see next slide)



```
<main class="container">
  <div class="row">
    <div class="col-12">Column 1</div>
    <div class="col-12">Column 2</div>
  </div>
  <div class="row">
    <div class="col-md-4">Column 1</div>
    <div class="col-md-8">Column 2</div>
  </div>
  <div class="row">
    <div class="col-md-4 col-sm-6">Column 1</div>
    <div class="col-md-8 col-sm-6">Column 2</div>
  </div>
</main>
```

Grid – CSS

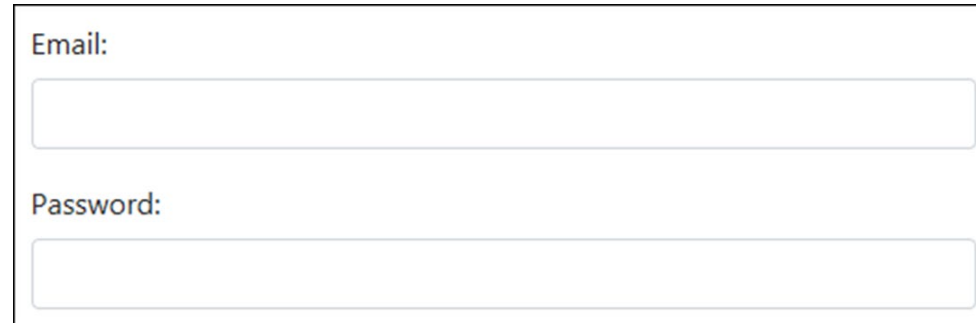
This is a custom CSS Class that overrides the Bootstrap CSS Class

To access the site.css file, navigate to `wwwroot > css > site.css`

```
.row {                                /* custom row styles */
    margin-bottom: 0.5rem;
}
.row div {                            /* custom column styles */
    border: 1px solid black;
    padding: 0.5rem;
    background-color: aliceblue;
}
```

Log In Samples

- A form with two text boxes in vertical layout



Email:

Password:

```
<form asp-action="Login" method="post">
  <div class="form-group">
    <label for="email">Email:</label>
    <input type="email" id="email" class="form-control" />
  </div>
  <div class="form-group">
    <label for="pwd">Password:</label>
    <input id="pwd" type="password" class="form-control" />
  </div>
</form>
```

Log In Samples

- A form with two text boxes in horizontal layout



Email:

Password:

```
<form asp-action="Login" method="post">
  <div class="form-group row">
    <label for="email" class="col-sm-2">Email:</label>
    <div class="col-sm-10">
      <input type="email" id="email" class="form-control" />
    </div>
  </div>
  <div class="form-group row">
    <label for="pwd" class="col-sm-2">Password:</label>
    <div class="col-sm-10">
      <input type="password" id="pwd" class="form-control" />
    </div>
  </div>
</form>
```

Navigation Samples - Breadcrumbs

[Home](#) / [Products](#) / Guitars

```
<nav aria-label="breadcrumb">
  <ol class="breadcrumb">
    <li class="breadcrumb-item">
      <a href="/">Home</a>
    </li>
    <li class="breadcrumb-item">
      <a href="/Products">Products</a>
    </li>
    <li class="breadcrumb-item active"
      aria-current="page">Guitars
    </li>
  </ol>
</nav>
```

Navigation Samples – Nav links styled as tabs



```
<nav class="nav nav-tabs">
  <a class="nav-item nav-link active" href="/">Home</a>
  <a class="nav-item nav-link" href="/products">
    Products</a>
  <a class="nav-item nav-link" href="/cart">Cart</a>
</nav>
```

Navigation Samples – The same nav links styled as pills



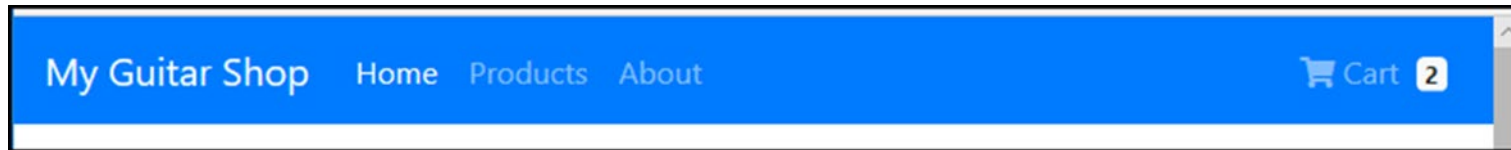
```
<nav class="nav nav-pills">
  <a class="nav-item nav-link active" href="/">Home</a>
  <a class="nav-item nav-link" href="/products">
    Products</a>
  <a class="nav-item nav-link" href="/cart">Cart</a>
</nav>
```


Navigation Samples – A more verbose way of coding the same nav links



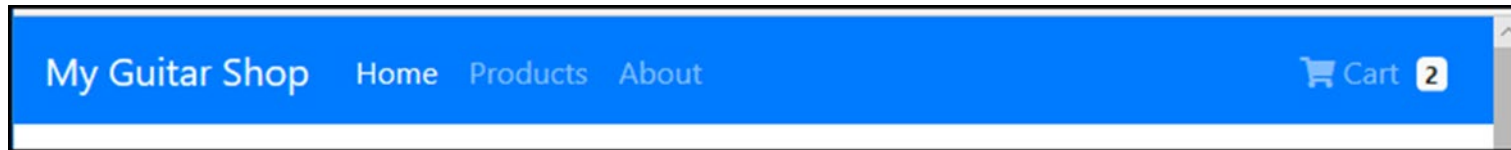
```
<ul class="nav nav-pills">
  <li class="nav-item">
    <a class="nav-link active" href="/">Home</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="/products">Products</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="/cart">Cart</a>
  </li>
</ul>
```

Navigation Samples – Navbar



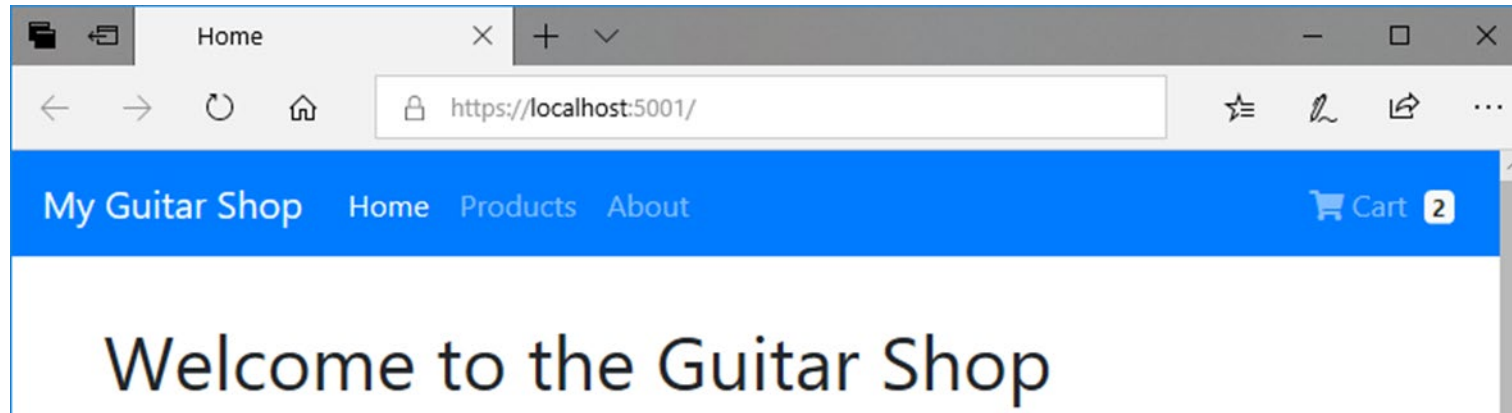
```
<nav class="navbar navbar-expand-md navbar-dark bg-primary">
  <a class="navbar-brand" href="/">My Guitar Shop</a>
  <button class="navbar-toggler" type="button"
    data-toggle="collapse"
    data-target="#navbarSupportedContent"
    aria-controls="navbarSupportedContent"
    aria-expanded="false"
    aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
  </button>
  <nav class="collapse navbar-collapse"
    id="navbarSupportedContent">
    <div class="navbar-nav mr-auto">
      <a class="nav-item nav-link active" href="/">Home</a>
      <a class="nav-item nav-link"
        href="/products">Products</a>
      <a class="nav-item nav-link" href="/about">About</a>
    </div>
```

Navigation Samples – Navbar cont.



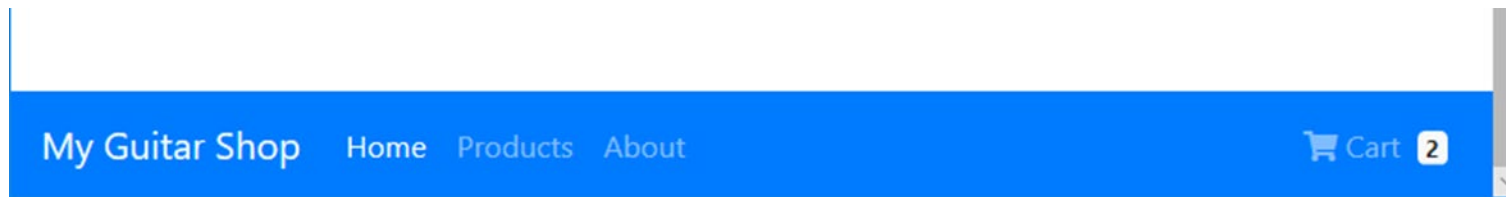
```
<div class="navbar-nav navbar-right">
  <a class="nav-item nav-link" href="/cart">
    <span class="fas fa-shopping-cart"></span>
    &nbsp;Cart&nbsp;
    <span class="badge badge-primary">2</span>
  </a>
</div>
</nav>
</nav>
```

Navbar Positioning – Top of the page



```
<body>
  <nav class="navbar navbar-expand-md navbar-dark
        bg-primary fixed-top">
    <!-- navbar items go here -->
  </nav>
  <div class="container">
    <!-- container items go here -->
  </div>
</body>
```

Navbar Positioning – Bottom of the page



```
<body>
  <nav class="navbar navbar-expand-md navbar-dark
          bg-primary fixed-bottom">
    <!-- navbar items go here -->
  </nav>
  <div class="container">
    <!-- container items go here -->
  </div>
</body>
```

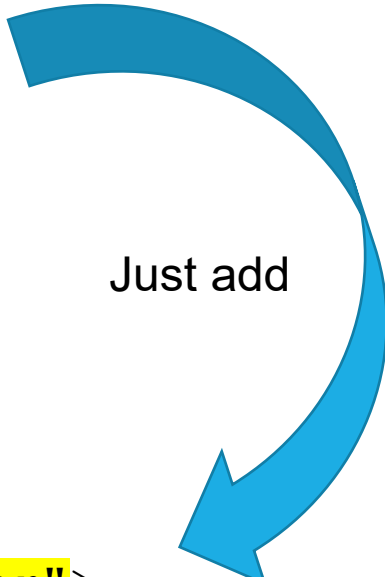
Table (Default)

Department	Phone Number	Extension
General	555-555-5555	1
Customer Service	555-555-5556	2
Billing and Accounts	555-555-5557	3

```
<table class="table">
  <thead>
    <tr>
      <th>Department</th>
      <th>Phone Number</th>
      <th>Extension</th></tr>
    </thead>
    <tbody>
      <tr>
        <td>General</td>
        <td>555-555-5555</td>
        <td>1</td></tr>
      <tr>
        <td>Customer Service</td>
        <td>555-555-5556</td>
        <td>2</td></tr>
      <tr>
        <td>Billing and Accounts</td>
        <td>555-555-5557</td>
        <td>3</td></tr>
    </tbody>
  </table>
```

Table with stripes and borders

Department	Phone Number	Extension
General	555-555-5555	1
Customer Service	555-555-5556	2
Billing and Accounts	555-555-5557	3



Just add

```
<table class="table table-striped table-bordered table-hover">...
```

Text Samples - Basic

This text is RIGHT-ALIGNED.

This Text Is Centered.

This text is left-aligned.

```
<p class="text-right">  
  This text is <span class="text-uppercase">  
    right-aligned</span>.  
</p>  
<p class="text-center">  
  <span class="text-capitalize">This text is centered.  
  </span>  
</p>  
<p class="text-left">  
  This text is <span class="text-lowercase">  
    LEFT-ALIGNED</span>.  
</p>
```


Text Sample – Success Class

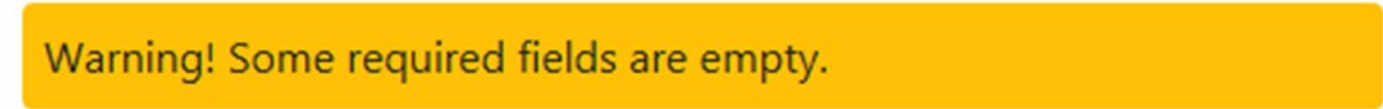
The success class applied to the text of an element

Congratulations! You are now registered.

```
<p class="text-success">  
    Congratulations! You are now registered.  
</p>
```

Text Sample – Warning Class

The warning class applied to the text of an element



Warning! Some required fields are empty.

```
<p class="bg-warning p-2 rounded">  
    Warning! Some required fields are empty.  
</p>
```

Text Sample – Dismissible Alert



```
<div class="alert alert-success alert-dismissible">
  <button class="close" data-dismiss="alert">
    &times;
  </button>
  Success! <a href="#" class="alert-link">
    Learn more</a>
</div>
```

Demo

Code Demos

- **Demo 1 – File:** BootstrapSampleCodes
- **Demo 2 - File:** Ch02FutureValue
- **Demo 3 – File:** Ch03FutureValue
- **Demo 4 –** Examples of Bad Web-app User Interfaces (Views)
- **Demo 5 –** Where to get the Murach Resources

Tasks

For this weeks Lab

1. An installed/set-up version of Visual Studio 2022 on your Personal Machine

For next weeks Lab

1. Draft wireframes for your User Interface
2. Formalised Team and documentation

For your Assignments

1. Formalising a team
2. Deciding on team roles
3. Wireframes for your user interfaces

Assignments due this week

1. None

Assignments due next week

1. Start-Up SCRUM & GitCommit - Structure

Optimistic developer: "This is a common sense feature that doesn't need documentation"

End user:



Any last minute questions?

INFT3050 – Web Programming